Layered automata: A canonical model for automata over infinite words

Antonio Casares, Christof Löding and Igor Walukiewicz

Abstract

We introduce layered automata, a subclass of alternating parity automata that generalises deterministic automata. Assuming a consistency property, these automata are history deterministic and 0-1 probabilistic. We show that every omega-regular language is recognised by a unique minimal consistent layered automaton, and that this canonical form can be computed in polynomial time from every layered or deterministic automaton. We further establish that, for layered automata, both consistency checking and inclusion testing can be performed in polynomial time. Much like deterministic finite automata, minimal consistent layered automata admit a characterisation based on congruences.

ACM Reference Format:
Antonio Casares, Christof Löding and Igor Walukiewicz. 2026. Layered automata: A canonical model for automata over infinite words. In *Proceedings of Submission to LICS'26*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

This document contains hyperlinks. Each occurrence of a notion is linked to its *definition*. On an electronic device, the reader can click on words or symbols (or just hover over them on some PDF readers) to see their definition.

1 Introduction

Automata over infinite words (ω-automata in the following) were first introduced by Büchi to show the decidability of monadic second-order logic over $(\mathbb{N}, <)$ [10]. Since then, they have found numerous applications in the verification and synthesis of reactive systems [15, 32, 40].

A key limitation of current ω-automata theory is the lack of minimal canonical models. Minimisation is instrumental for practical applications, particularly when automata are derived from specifications, like LTL formulas, through hierarchical translations. In this process, each constructor in the specification corresponds to an operation on the automaton – some of which necessitate a powerset construction. As we know from the case of finite words and trees, minimisation after each operation is key to efficient translations [30]. Minimisation is also important for theory. One immediate example is learning algorithms for ω-regular languages. More broadly, achieving polynomial-time minimisation indicates that we have gained insights into the structural properties of acceptors. These insights may eventually allow us to revisit long-standing open problems, for instance the index problem for tree automata [17, 28].

Contrary to the case of automata over finite words, deterministic ω-automata do not have minimal canonical models, as a language may admit several non-isomorphic deterministic automata with a minimal number of states. Moreover, the minimisation problem is NP-hard for all kinds of deterministic ω-automata expressive enough to recognise all ω-regular languages [1, 11, 41]. While there are some canonical ways to represent ω-regular languages, such as ω-semigroups or congruences, these representations are potentially exponentially larger than deterministic parity automata, and furthermore, they cannot be used directly in verification and synthesis applications.

History deterministic (HD) automata are non-deterministic automata for which the non-determinism can be resolved on the fly. They were originally introduced under the name of *good for games* [25][1], as they are exactly those automata which can be composed with any game while preserving the winner. This property allows their use in applications in verification and synthesis instead of deterministic automata. Every deterministic automaton is history deterministic, but a history deterministic automaton can be exponentially smaller than the smallest deterministic one [31]. History determinism has been generalised to alternating ω-automata [9, 16]. Alternating history deterministic automata have the same good compositionality properties making them suitable for applications in verification and synthesis, while being even smaller than non-deterministic HD automata [8, Lemma 6].

Other types of automata that are actively studied are those in which the non-determinism can be resolved randomly [5, 26, 38], or automata that compose faithfully with MDPs [24]. In particular, *0-1 probabilistic automata* are automata in which a random walk over an infinite word is accepting with probability 1 if and only if the word belongs to the semantics of the automaton; otherwise, it is rejecting with probability 1. Such automata are in particular suitable for the study of MDPs [26].

In this work, we propose a new model of acceptors for ω-regular languages that we call layered automata. Our starting point is the result of Abu Radi and Kupferman [2] stating that (transition-based) history deterministic coBüchi automata can be minimised in polynomial time and admit a canonical minimal representation. It has been further shown that this canonical automaton can be described in a natural way using a congruence over pairs of finite words [34]. However, HD coBüchi automata can only recognise a fragment of ω-regular languages: those that are in the Σ^0_2 level in the Borel hierarchy, or equivalently, persistent properties in the Manna-Pnueli hierarchy [36].

The model of layered automata that we propose in this work can be seen as an extension of the minimisation result of Abu Radi and Kupferman to all ω-regular languages. This is in line with some recent works such as *chains of coBüchi automata* [22] and *rerailing automata* [19] which we compare with in Section 3.6. Orthogonally, layered automata can also be seen as a generalisation of the Zielonka tree of a Muller language [43] and of signature automata for positional languages [13]. We provide ample evidence that layered automata have good structural and algorithmic properties.

[1] The related notion of *good for trees automata* was previously introduced in [33].

Submission to LICS'26, Lisbon, Portugal
2026. ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Contributions

We introduce *layered automata*, a formalism to represent all ω-regular languages. A layered automaton can be understood as a representation of an alternating parity automaton with some good structural properties. If a layered automaton satisfies an additional property of *consistency*, then the associated alternating automaton is history deterministic.

Consistent layered automata combine the structure of both minimal history deterministic coBüchi automata and Zielonka trees. They also encompass deterministic parity automata. They possess several remarkable properties. Most notably, they admit a canonical minimal form that is computable in polynomial time. This canonicity result can be stated as follows:

Theorem 1.1. *Every ω-regular language L can be recognised by a consistent layered automaton $\mathcal{A}_L$ such that every equivalent consistent layered automaton contains a subautomaton admitting a surjective morphism to $\mathcal{A}_L$. Moreover, $\mathcal{A}_L$ can be computed in polynomial time from any given consistent layered automaton recognising L.*

We detail further properties of layered automata.

Applicability. Consistent layered automata are representations of alternating parity automata that are guaranteed to be both history deterministic and 0-1 probabilistic (Propositions 3.13 and 3.14). Therefore, consistent layered automata have the key properties that make them suitable for applications in verification and synthesis. Moreover, they provide an easy-to-check sufficient condition for HDness and being 0-1 probabilistic, as layered automata are defined syntactically and checking consistency can be done in polynomial time (Proposition 3.15).

As layered automata generalise minimal coBüchi automata, they can be exponentially smaller than minimal deterministic parity automata [31].

Emptiness and inclusion of consistent layered automata can be done in polynomial time (Propositions 3.16 and 3.17).

Canonicity, morphisms, and congruences. We show that every ω-regular language can be recognised by a unique minimal consistent layered automaton. More precisely, we show that for every ω-regular language L, there is a consistent layered automaton $\mathcal{A}_L$ that is normal, centralised and safe minimal and any layered automaton with these three properties is isomorphic to $\mathcal{A}_L$ (Theorem 4.1). Moreover, every consistent layered automaton recognising L contains a subautomaton that admits a surjective morphism to $\mathcal{A}_L$ (Theorem 5.1). Furthermore, we provide a characterisation in terms of congruences for the minimal layered automaton $\mathcal{A}_L$ (Theorem 6.6). This characterisation is purely based on the language L, and does not depend on a given representation as an automaton. To the best of our knowledge, these are the first results of this type for an automaton model capable to recognise all ω-regular languages and having the minimal form never bigger than deterministic parity automata.

Minimisation in PTIME. We present an algorithm that, given a consistent layered automaton $\mathcal{A}$, computes in polynomial time the minimal layered automaton for $L(\mathcal{A})$ (Theorem 5.1). Since every deterministic parity automaton is trivially a consistent layered automaton, this yields an efficient minimisation procedure transforming deterministic parity automata into layered automata. This does not contradict the above-mentioned lower bounds on minimisation, as the resulting automaton is alternating.

Relation to other models. We provide some examples and indicate some results comparing and relating layered automata to other models such as chains of coBüchi automata and Zielonka trees. We also discuss how we can use layered automata to characterise important properties of ω-regular languages, such as positionality, following [13]. While we leave a more detailed study of all these relations as future work, our discussion shows that layered automata have many interesting connections to various other well-established and more recent formalisms from the theory of ω-regular languages.

Related work

As mentioned, the starting point of our work is the minimisation procedure for HD coBüchi automata from Abu Radi and Kupferman [2]. By duality, this entails a minimisation procedure for universal HD Büchi automata, and the notion of layered automata can be seen as a natural extension to arbitrary parity indices that mixes nondeterministic and universal branching. Layered automata are also inspired by signature and ε-complete automata from [13, Sect. 3.3], as well as the minimisation procedure for them. The proof that all layered automata are history deterministic generalises ideas from [31, Lemma 3]. The congruence-based characterisation is an extension of the congruences for coBüchi automata from [34, Sect 3], using conditions closely related to the priority mapping on finite words defined in [7, Def 3.1,3.5].

Various representations of ω-regular languages admitting – in some sense – canonical models exist in the literature. These include some algebraic approaches based on congruences, as Arnold's congruence [4], Wilke-algebras and ω-semigroups [39, 42], families of right congruences [35, Def. 5], and families of DFAs [3, 29]. These objects have the significant disadvantage that they can be exponentially larger than deterministic parity automata (it is folklore that already for regular languages of finite words, the syntactic monoid can be of size n^n for the minimal DFA being of size n, see [27, Thm. 3] for a concrete example; this exponential gap transfers to all the aforementioned formalisms). While their size is incomparable with the size of a smallest deterministic parity automata (DPA) for a language, they cannot directly be used in applications like verification or synthesis.

Another representation, taking the form of a specific DPA, is proposed in [7, Sect. 3]. There, the "precise DPA" of a language is defined as a product of deterministic Mealy machines that are extracted directly from the language in terms of a coloring function. However, the precise DPA can be exponentially larger than minimal DPA.

More recently, some canonical representations based on HD automata have been introduced. In 2022, Ehlers and Schewe proposed to represent an ω-regular language as a decreasing chain of coBüchi-recognisable languages [22]. Each of these languages can be recognised by the minimal HD coBüchi automaton from Abu Radi and Kupferman [2], leading to a canonical representation. This formalism has subsequently been studied under the name *COCOA* [20, 21]. Minimal layered automata are clearly related to COCOA, but there are some important differences. A key difference

is that layered automata bring a direct connection with alternating history deterministic automata—which are widely accepted as the class of ω-automata suitable for verification and synthesis. Notably, we conjecture that canonical layered automata are minimal amongst all history deterministic alternating parity automata (Conjecture 7.1). The size comparison between both models is somewhat subtle. While COCOA can be smaller than layered automata, in order to obtain an HD automaton from a COCOA, one must take a product of all its components [20]. We give a more detailed comparison with COCOA in Section 3.6.

In 2025, Ehlers introduced *rerailing automata* [19, Def. 1] and shows that they can be minimised efficiently. These are automata where each run is associated with a priority (the minimal appearing infinitely often), and a word is accepted if the maximal priority of its runs is even. Moreover, they must satisfy a semantic "rerailing property", inspired from the properties of minimal HD coBüchi automata. In [19, Sect. VI], rerailing automata are shown to be usable in reactive synthesis, but their exact relation with HD automata is not yet established.

Appendix. The appendix contains the full version of the paper with more examples, explanations, and complete proofs. The full version follows the same structure into sections. Whenever needed, we provide specific references to the full version.

2 Preliminaries

We let $[d] = \{1, \dots, d\}$, and we call the elements of $[d]$ *priorities*. We use the variables x, y to denote priorities. The (min) *parity condition* is the language over $[d]^\omega$ given by:

$$\mathsf{Parity} = \{x_1 x_2 \cdots \in [d]^\omega \mid \liminf x_i \text{ is even}\}.$$

Transition systems. A *transition system* over a set Σ is an edge-labelled directed graph $\mathcal{T} = (Q, \Delta)$, where $\Delta \subseteq Q \times \Sigma \times Q$. If $(q, a, q') \in \Delta$, we also write $q \xrightarrow{a} q'$. It is *complete* if for all $q \in Q$ and $a \in \Sigma$, there is some $(q, a, q') \in \Delta$. It is *deterministic* if for every $q \in Q$ and $a \in \Sigma$ there is at most one q' such that $(q, a, q') \in \Delta$. We prefer to represent the transitions of deterministic transition systems by a function $\delta \colon Q \times \Sigma \to Q \cup \{\bot\}$, where $\delta(q, a) = q'$ if $(q, a, q') \in \Delta$, and $\delta(q, a) = \bot$ if no such transition exists.

For a word $w \in \Sigma^*$ we write $q \xrightarrow{w} q'$ if there is a path from q to q' whose label is w, which we call a *run*. We sometimes omit q', writing just $q \xrightarrow{w}$, if q' is irrelevant. We write $q \xrightarrow{w} \bot$ it there is no path with label w from q.

If $\mathcal{T}$ is a transition system over a product alphabet $\Sigma \times [d]$, by an abuse of notation we call a *run over* $w \in \Sigma^\omega$ any path having w as labels in the Σ-component. In this case, we also say that a run *is accepting* if its sequence of $[d]$-labels satisfies the parity condition. We write $q \xrightarrow{a:x} q'$ if $(q, (a, x), q') \in \mathcal{T}$, and $q \xrightarrow{a:\geq x} q'$ if $(q, (a, y), q') \in \mathcal{T}$ for some $y \geq x$, similarly for $> x$, etc. We write $q \xrightarrow{w:\geq x} q'$ if the minimal priority over the path is $\geq x$, similarly for $> x$, etc.

A *morphism* between two transition systems $\mathcal{T} = (Q, \Delta), \mathcal{T}' = (Q', \Delta')$ over the same set Σ is a function $\mu \colon Q \to Q'$ such that if $(p, a, q) \in \Delta$ then $(\mu(p), a, \mu(q)) \in \Delta'$. Note that any run over w in $\mathcal{T}$ from p maps to a run over w in $\mathcal{T}'$ from $\mu(p)$.

Deterministic parity automata. A *deterministic parity automaton* is a complete deterministic transition system $\mathcal{A} = (Q, \Delta)$ over the set $\Sigma \times [d]$ with a designated initial state $q_{\mathsf{init}} \in Q$.

A word $w \in \Sigma^\omega$ is accepted by the automaton if the unique run over w satisfies the parity condition. The language of the automaton is the set of words it accepts. A language is *ω-regular* if it can be recognised by a deterministic parity automaton.

Simple by priorities alternating parity automata. In full generality, an *alternating parity automaton* is given by a set of states together with a transition function of the form $\delta \colon Q \times \Sigma \to \mathsf{Bool}^+(Q \times [d])$. In this work, all alternating automata are of a simpler form, which we call *simple by priorities alternating automata*. These correspond to automata where the current state and letter determine both which player chooses the next transition and the priority of the transition.

A *simple by priorities* automaton is a complete transition system $\mathcal{A} = (Q, \Delta)$ over the set $\Sigma \times [d]$ with a designated *initial state* $q_{\mathsf{init}} \in Q$, satisfying that for every $q \in Q$ and $a \in \Sigma$ all a-transitions from q have the same priority. So there is x a *priority of action* a in q, meaning that for all $(q, (a, y), q') \in \Delta$ we have $y = x$.

The semantics of a simple by priorities automaton is defined via game where the priority of an action determines who makes the move. Given an input word $w = a_1 a_2 \cdots \in \Sigma^\omega$, we define the game $\mathcal{G}(\mathcal{A}, w)$ where two players, called Eve and Adam, build a run over w in $\mathcal{A}$ as follows:

- The initial position is the state $q_1 = q_{\mathsf{init}}$.
- Suppose the current run constructed in a play ends in a state q_i, and x_i is the priority of a_i in q_i. If x_i is odd then Eve picks a transition $q_i \xrightarrow{a_i:x_i} q_{i+1}$, otherwise Adam makes the choice.
- Eve wins if the constructed path is accepting.

We say that w is *accepted* by $\mathcal{A}$ if Eve has a winning strategy in the game $\mathcal{G}(\mathcal{A}, w)$. Note that, by determinacy of parity games [37], on the contrary Adam has a winning strategy. The *language recognised* by $\mathcal{A}$, written $L(\mathcal{A})$, is the set of words it accepts. The *language of a state* $q \in Q$ is the language recognised by the automaton obtained by fixing q as the initial state. We denote it $L(\mathcal{A}, q)$, or just $L(q)$ if $\mathcal{A}$ is clear from the context.

Semantic determinism. An automaton $\mathcal{A}$ is *semantically deterministic* if whenever we have $p \xrightarrow{a} q_1$ and $p \xrightarrow{a} q_2$ then $L(\mathcal{A}, q_1) = L(\mathcal{A}, q_2)$. Equivalently, if $p \xrightarrow{a} q$ then $L(\mathcal{A}, q) = a^{-1}L(\mathcal{A}, p)$.

Remark 1. In a semantically deterministic automaton: if $q_{\mathsf{init}} \xrightarrow{u} q$ then $L(q) = u^{-1}L = \{w \in \Sigma^\omega \mid uw \in L\}$.

Resolvers and history determinism. Consider a simple by priorities automaton $\mathcal{A}$. A *resolver* for Eve in $\mathcal{A}$ is a function $\sigma \colon (\{q_{\mathsf{init}}\} \cup \Delta^+) \times \Sigma \to Q$, satisfying that whenever a run $\rho \in (\{q_{\mathsf{init}}\} \cup \Delta^+)$ ends in a state q, and the priority of a in q is odd then $q' = \sigma(\rho, a)$ is an a-successor of q, namely $q \xrightarrow{a} q'$. A run is *consistent with* σ (also called a *σ-run*) if for every prefix $\rho \cdot (q, (a, x), q')$ with x odd, we have $\sigma(\rho, a) = q'$. The *language accepted by an Eve-resolver* σ is

$$L(\mathcal{A}, \sigma) = \{w \in \Sigma^\omega \mid \text{ every } \sigma\text{-run over } w \text{ is accepting}\}.$$

Note that we always have $L(\mathcal{A}, \sigma) \subseteq L(\mathcal{A})$. We say that the Eve-resolver σ is *valid* if $L(\mathcal{A}, \sigma) = L(\mathcal{A})$. We define symmetrically

resolvers for Adam and their languages (note that if τ is an Adam-resolver, then $L(\mathcal{A},\tau) \supseteq L(\mathcal{A})$). For two fixed resolvers σ and τ for Eve and Adam, respectively, every word admits a unique run that is consistent with both σ and τ. We write $L(\mathcal{A},\sigma,\tau)$ for the set of words for which this run is accepting.

An alternating automaton is *history deterministic* (HD) if both Eve and Adam have valid resolvers. That is, if there are resolvers σ and τ for Eve and Adam such that

$$L(\mathcal{A}) = L(\mathcal{A},\sigma) = L(\mathcal{A},\tau) = L(\mathcal{A},\sigma,\tau).$$

3 Layered automata

In this section we introduce layered automata, the central notion of this paper. We develop their definition in two stages: the syntactic structure of layered automata (Section 3.1), and the language they recognise via an associated alternating parity automaton (Section 3.2). Thanks to the structure of a layered automaton the associated automaton is guaranteed to be simple by priorities.

Next, we introduce uniform semantic determinism, a slightly stronger notion than semantic determinism. Semantic determinism is a necessary condition for an automaton to be HD. For layered automata, uniform semantic determinism yields two significant consequences: the associated alternating automaton is both history deterministic and 0-1 probabilistic (Propositions 3.13 and 3.14).

Our main technical tool, presented in Section 3.3, is a characterisation of acceptance by layered automata that does not rely on alternation. For this we introduce a notion of consistency, which characterises uniform semantic determinism. For layered automata, we can test consistency in polynomial time (Proposition 3.15). Furthermore, for consistent layered automata, emptiness and language inclusion are decidable in polynomial time (Propositions 3.16 and 3.17).

3.1 Syntactic definition of layered automata

A *layered automaton* $\mathcal{A}$ is a tuple of deterministic transition systems $\mathcal{T}_1,\dots,\mathcal{T}_d$ together with morphisms $\mu_x : \mathcal{T}_{x+1} \to \mathcal{T}_x$, for $1 \le x < d$. Additionally, we require that $\mathcal{T}_1$ is complete, has a distinguished initial state q_{init} in $\mathcal{T}_1$, and all states of $\mathcal{T}_1$ are reachable from q_{init}.[2] Below we introduce some notions allowing to talk about the structure of layered automata, and give some intuition behind it.

We say that $\mathcal{T}_x$ is the *x-th layer* of the automaton. We use Q_x to denote the set of states of $\mathcal{T}_x$, which we refer to as *x-states*. We assume $Q_1,\dots,Q_d$ pairwise disjoint. We let $\delta_x : Q_x \times \Sigma \to Q_x \cup \{\bot\}$ be the (partial) transition function of $\mathcal{T}_x$ (which is required to be total for $\mathcal{T}_1$). We write $p \xrightarrow{a}_x q$ to emphasize that a transition is taken in the x-th layer of the automaton. Sometimes we write Q_{x+1} for x some layer; we assume that this is the empty set when $x = d$.

We write $Q_{\ge x}$ for the union of $Q_x,\dots,Q_d$, so $Q_{\ge 1}$ is the union of all the sets of states.

For every x, we define $\widehat{\mu}_x : Q_{\ge x} \to Q_x$ by

$$\widehat{\mu}_x(q) = q, \text{ for } q \in Q_x \;\; ; \;\; \text{and}$$
$$\widehat{\mu}_x(q) = \mu_x(\mu_{x+1}(\dots\mu_{y-1}(q)\dots)), \text{ for } q \in Q_y,\ y > x$$

For a state q, let *layer*(q) be the layer x such that $q \in Q_x$. For a pair (q,a) of a state and a letter, let *layer*(q,a) be the biggest layer x such that $\delta_x(\widehat{\mu}_x(q),a)$ is defined. Observe that *layer*(q,a) is always defined as $\mathcal{T}_1$ is complete.

We say that p is an *ancestor* of q if $\widehat{\mu}_x(q) = p$, for $x = \mathit{layer}(p)$. Given $q,p \in Q_{\ge x}$, we write $q \sim_x p$ if $\widehat{\mu}_x(q) = \widehat{\mu}_x(p)$ and we let $[q]_x = \widehat{\mu}_x^{-1}(\widehat{\mu}_x(q))$. A state $q \in Q_{\ge 1}$ is a *leaf state* if it is not an image of any μ_x.

The forest of a layered automaton. We can view a layered automaton as a *forest* with nodes $Q_{\ge 1}$. Each state in $\mathcal{T}_1$ is the root of a tree, a node $q \in Q_{x+1}$ has $\mu_x(q) \in Q_x$ as the parent. The levels of this forest are labelled from 1 (roots) to d.

In this way, *layer*(q) is the level of q in this forest. The function $\widehat{\mu}_x(q)$ designates the ancestor at level x of a node $q \in Q_{\ge x}$. We have that $q \sim_x p$ if p and q have the same ancestor at level x. Equivalently, $[q]_x$ is the set of nodes of the subtree rooted at the ancestor of q at level x. As expected, leaf states are the states that are leaves in this forest. Observe that more than one level can have leave states.

Example 3.1. In Figure 1 shows a layered automaton represented as tree. Each transition system constitutes a level of the tree. The morphisms μ_x, corresponding to the parent relation, are represented as dotted arrows.

In this example, all leaf states are on the same layer. Figure 4 (Section 6, Example 6.1) shows a layered automaton with leaf states at different layers.

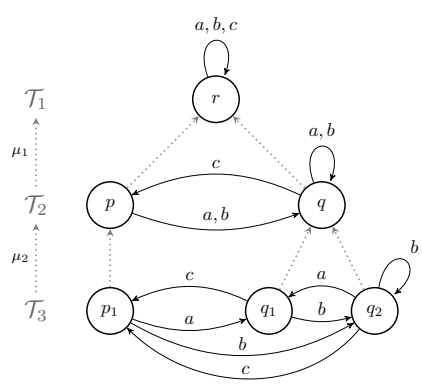

Figure 1: A layered automaton represented as a tree.

3.2 Semantics via alternating parity automata

We now define acceptance for layered automata by associating to each layered automaton $\mathcal{A}$ an alternating automaton $[\![\mathcal{A}]\!]$, called the semantics of $\mathcal{A}$; the language of $\mathcal{A}$ is defined as the language of $[\![\mathcal{A}]\!]$. The tree structure of $\mathcal{A}$ constrains the transitions of $[\![\mathcal{A}]\!]$ to be simple by priorities. With this semantics in place, we introduce uniform semantic determinism, a slight strengthening of semantic determinism. This property ensures, that all leaf states with the same ancestor in layer 1 accept the same languag—a constraint we later characterise via consistency (Section 3.3).

For a given layered automaton $\mathcal{A}$ its *semantics automaton* $[\![\mathcal{A}]\!]$ is a simple by priorities alternating parity automaton defined as follows. The states Q are the leaf states of $\mathcal{A}$. The initial state is any

[2] The choice of starting at index 1 (instead of for instance 0) is arbitrary. It has been chosen so that layered automata with two layers correspond to the well understood class of HD coBüchi automata.

leaf state q with $\widehat{\mu}_1(q) = q_{\text{init}}$ the initial state in $\mathcal{T}_1$.[3] The transitions are

$$\Delta = \left\{ (q, a, x, q') \in Q \times \Sigma \times [d] \times Q \quad \middle| \quad \begin{array}{l} x = \mathit{layer}(q, a), \text{ and} \\ \widehat{\mu}_x(q) \xrightarrow{a} \widehat{\mu}_x(q') \text{ in } \mathcal{T}_x \end{array} \right\}.$$

Intuitively, when the automaton is in a leaf q and is given a letter a it looks at $\mathit{layer}(q, a) = x$, that is, the largest x such that the transition $\widehat{\mu}_x(q) \xrightarrow{a}_x p$ is defined. There is a transition from q to every leaf $q' \in \widehat{\mu}_x^{-1}(p)$. All these transitions have priority x. If x is odd, Eve chooses a leaf under p, otherwise it is Adam. In this case, we sometimes write $q \xrightarrow{a:x} [q']_x$.

In the following, we focus almost exclusively on layered automata such that $[\![\mathcal{A}]\!]$ is semantically deterministic. We actually work with a slightly stronger property.

Definition 3.2. A layered automaton $\mathcal{A}$ is *uniformly semantically deterministic* if for every pair of leaf states $p \sim_1 q$ we have $L([\![\mathcal{A}]\!], p) = L([\![\mathcal{A}]\!], q)$.

In Proposition 3.15 we show that we can check in polynomial time if a given layered automaton is uniformly semantically deterministic.

Remark 2. If $\mathcal{A}$ is uniformly semantically deterministic then $\mathcal{T}_1$ admits a morphism into the *automaton of residuals* of $L = L([\![\mathcal{A}]\!])$: the transition system with states the sets $u^{-1}L$ and transitions $u^{-1}L \xrightarrow{a} (ua)^{-1}L$. We rely here on our assumption that all states in $\mathcal{T}_1$ are reachable from q_{init}.

Example 3.3. In Figure 2, we show the alternating parity automaton corresponding to the layered automaton of Figure 1. It recognises the language

$$L = \text{Words with finitely many } cc \text{ and infinitely many } aa.$$

The states are the leaves of the tree. Whenever a letter c is seen from state p_1, priority 1 is produced, and Eve can choose to go to any state. Whenever a letter a is seen from state q_1, priority 2 is produced, and Adam can choose to go to q_1 or q_2. The rest of transitions produce priority 3, and correspond to the transitions at level 3 in the layered automaton.

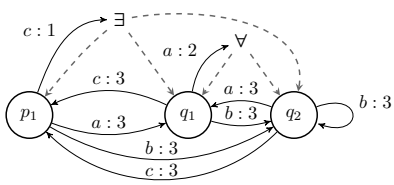

Figure 2: Alternating parity automaton for the layered automaton of Figure 1.

Deterministic parity automata as layered automata. Deterministic automata can be seen as a special case of layered automata. Consider a deterministic parity automaton $\mathcal{B}$. For every priority x, let $\mathcal{B}_{\geq x}$ be the transition system obtained by restricting $\mathcal{B}$ to transitions with priorities $\geq x$. Let $\mathcal{A}$ be the layered automaton with transition systems $\mathcal{T}_x = \{x\} \times \mathcal{B}_{\geq x}$ and morphisms $\mu_x((x+1, q)) = (x, q)$ for all priorities x and states q. (Note that the x-component is just added for obtaining disjointness of the sets Q_x.) It is direct to check that $[\![\mathcal{A}]\!] = \mathcal{B}$.

Minimal HD coBüchi automata. A nondeterministic *coBüchi automaton* is a parity automaton (in the classical sense) using priorities $\{1, 2\}$. In [31] Kuperberg and Skrzypczak showed that history deterministic coBüchi automata can be pruned not only into semantically deterministic, but also into *safe deterministic* automata at the same time. The later property meaning that the non-deterministic choices only appear in transitions carrying priority 1. Such automata are simple by priorities automata with priorities $\{1, 2\}$ as for every $q \in Q$ and $a \in \Sigma$ there is at most one transition $q \xrightarrow{a:2} q'$.

It is easy to see that every HD coBüchi automaton can be made 1-*saturated*, meaning that whenever $q \xrightarrow{a:1} p$, then $q \xrightarrow{a:1} p'$ for all p' with $L(p') = L(p)$.

We can observe that if $\mathcal{B}$ be a semantically deterministic, safe deterministic, 1-saturated coBüchi automaton then there is a layered automaton $\mathcal{A}$ such that $[\![\mathcal{A}]\!] = \mathcal{B}$. We take $\mathcal{T}_1$ to be the automaton of residuals of $\mathcal{B}$ and $\mathcal{T}_2 = \mathcal{B}_{\geq 2}$, with $\mu_1(q) = L(q)$. Abu Radi and Kupferman showed that HD coBüchi automata can be minimised in polynomial time. This minimisation is a special case of the minimisation of layered automata presented here.

3.3 Semantics via safe languages

This section offers a more direct approach to defining the language of a layered automaton. We introduce a restriction called consistency directly on layered automata, which we prove equivalent to uniform semantic determinism. We show that for consistent automata, we can characterise the language of $[\![\mathcal{A}]\!]$ through an acceptance condition that does not rely on alternation. The new acceptance condition closely parallels the natural color of words [22] and the acceptance mechanisms of rerailing automata [19], as discussed further in Section 3.6.

Safe languages. The *x-safe language* of a state $q \in Q_{\geq x}$ is

$$L_x(q) = \{w \in \Sigma^* \cup \Sigma^\omega \mid \text{ the is a run over } w \text{ from } \widehat{\mu}_x(q) \text{ in } \mathcal{T}_x\}.$$

That is, $L_x(q)$ are the words on which there is a path from the ancestor of q at level x. Note that by completeness of $\mathcal{T}_1$ we have $L_1(q) = \Sigma^* \cup \Sigma^\omega$ for every state q. When we need to specify the automaton $\mathcal{A}$, we write $L_x^{\mathcal{A}}(q)$.

Example 3.4. For the layered automaton of Figure 1, we have

$$L_2(q) = \text{Words without the factor } cc,$$
$$L_3(q_2) = \text{Words without the factors } cc \text{ and } aa.$$

The following equivalences relate the words labelling paths with no priority $< x$ in $[\![\mathcal{A}]\!]$ and x-safe languages. Recall that we write $\xrightarrow{w:\geq x}$ if the minimal priority on this path is $\geq x$. We have $w \in L_x(q)$ when:

- there is a path $\widehat{\mu}_x(q) \xrightarrow{w}_x$ in $\mathcal{T}_x$, or
- there is a path $[q]_x \xrightarrow{w:\geq x}$ in $[\![\mathcal{A}]\!]$, or

[3] Formally, this defines a family of automata. In general, these automata are not necessarily equivalent. However, for the automata considered in this paper (see notion of uniformly SD and consistency below), the choice of initial state is irrelevant.

- there is no path $[q]_x \xrightarrow{w}$ in $[\![\mathcal{A}]\!]$ seeing a priority $< x$.

Definition 3.5. Consider a layered automaton $\mathcal{A}$. A word $w \in \Sigma^\omega$ is *ultimately safe* in layer x if there is a decomposition $uw' = w$ and a state p on layer x such that $q_{\text{init}} \xrightarrow{u}_1 \widehat{\mu}_1(p)$ and $w' \in L_x(p)$.

Definition 3.6. A word $w \in \Sigma^\omega$ is *accepted* by $\mathcal{A}$ if the maximal layer in which w is ultimately safe is even. If it is odd then w is *rejected* by $\mathcal{A}$.

We aim at making a link between acceptance by $\mathcal{A}$ and acceptance by $[\![\mathcal{A}]\!]$. For this we need to define strong acceptance and consistency.

Strong acceptance. We say that $w \in \Sigma^\omega$ is *strongly accepted by p* if p is on some even layer x, $w \in L_x(p)$, and for every decomposition $w = uw'$, there is no $p' \in Q_{x+1}$ with $\widehat{\mu}_x(p) \xrightarrow{u}_x \widehat{\mu}_x(p')$ and $w' \in L_{x+1}(p')$. The notion of w being *strongly rejected* is defined similarly but for x odd.

Example 3.7. In the layered automaton from Figure 1, any word containing infinitely many factors cc's is strongly rejected by the state $r \in Q_1$. The state $q \in Q_2$ strongly accepts the words that contain no factor cc and infinitely many factors aa. Finally, the states on layer 3 strongly reject the words in their safe languages.

The next lemma gives some intuition behind this notion.

Lemma 3.8. *Let $\mathcal{A}$ be a layered automaton, p a leaf state. If a word w is strongly accepted by $\widehat{\mu}_x(p)$, for some even x, then $w \in L([\![\mathcal{A}]\!], p)$. Analogously, if w is strongly rejected by $\widehat{\mu}_x(p)$, for some odd x, then $w \notin L([\![\mathcal{A}]\!], p)$*

Proof. Every run over w from p produces the priority x infinitely often (regardless of the strategies of Eve and Adam). Indeed, since $w \in L_x(p)$, only priorities $\geq x$ can appear on a run over w from p. Every such a run must produce priority x infinitely often, as otherwise there would be a decomposition of the form $p \xrightarrow{u} q \xrightarrow{w':>x}$. □

In the other direction, if $[\![\mathcal{A}]\!]$ accepts a word then its suffix is strongly accepted from an appropriate state.

Lemma 3.9. *If $\mathcal{A}$ accepts w then there is a decomposition $uw' = w$ and a state p on some even layer x such that $q_{\text{init}} \xrightarrow{u}_1 \widehat{\mu}_1(p)$ and w' is strongly accepted from p.*

Proof. Consider the maximal layer x where w is ultimately safe, which is even as w is accepted. By definition, we have a decomposition $uw' = w$, with a state p, a path $q_{\text{init}} \xrightarrow{u}_1 \widehat{\mu}_1(p)$, and $w' \in L_x(p)$. It is direct to check that w' is strongly accepted from p, because otherwise x would not be the maximal layer where w is ultimately safe. □

A priori, a word can be both strongly accepted by some state, and strongly rejected by a different state. We forbid this by imposing an extra condition called consistency, defined below.

Consistency. We say that a layered automaton $\mathcal{A}$ is *consistent* if there is no pair of states $p, p' \in Q_{\geq 1}$ with $\widehat{\mu}_1(p) = \widehat{\mu}_1(p')$, and a word $w \in \Sigma^\omega$ such that w is strongly accepted by p and w is strongly rejected by p'.

As stated below, this notion is equivalent to uniform semantic determinism of $[\![\mathcal{A}]\!]$.

We state the key result saying that the semantics via alternating parity automata and via strong acceptance coincide for consistent layered automata. The proof, which requires introducing longest suffix resolvers in $[\![\mathcal{A}]\!]$, is included in Section 3.4 (Thm. 3.16) in the appendix.

Theorem 3.10. *Let $\mathcal{A}$ be a consistent layered automaton. A word $w \in \Sigma^\omega$ is accepted by $\mathcal{A}$ if and only if $w \in L([\![\mathcal{A}]\!])$. Moreover, $[\![\mathcal{A}]\!]$ is uniformly semantically deterministic.*

Corollary 3.11. *A layered automaton is consistent if and only if it is uniformly semantic deterministic.*

Indeed, the left to right implication is given by the theorem above. The other direction follows from the definitions and Lemma 3.8.

Corollary 3.12. *Let $\mathcal{A}$ be a consistent layered automaton and let $w \in \Sigma^\omega$. The following are equivalent:*

- *$w \in L([\![\mathcal{A}]\!])$,*
- *w is accepted by $\mathcal{A}$,*
- *There is a decomposition $uw' = w$ and a state p such that $q_{\text{init}} \xrightarrow{u}_1 \widehat{\mu}_1(p)$ and p strongly accepts w', and*
- *There is no decomposition $uw' = w$ and a state p such that $q_{\text{init}} \xrightarrow{u}_1 \widehat{\mu}_1(p)$ and p strongly rejects w'.*

3.4 Layered automata are history deterministic and 0-1 probabilistic

It turns out that the structure of a layered automaton guarantees interesting properties of $[\![\mathcal{A}]\!]$.

Proposition 3.13. *For every consistent layered automaton $\mathcal{A}$, the alternating parity automaton $[\![\mathcal{A}]\!]$ is history deterministic.*

We say that a simple by priorities parity automaton $\mathcal{A}$ is a *0-1 probabilistic automaton* if for every random walk ρ over a word $w \in \Sigma^\omega$ starting in q_{init}, it holds:

- $\Pr(\rho \text{ is accepting }) = 1 \iff w \in L(\mathcal{A})$, and
- $\Pr(\rho \text{ is accepting }) = 0 \iff w \notin L(\mathcal{A})$.

We note that 0-1 probabilistic automata are *good for MDPs* in the sense of [24] See [6, 26] for related notions.

Proposition 3.14. *For every consistent layered automaton $\mathcal{A}$, the alternating parity automaton $[\![\mathcal{A}]\!]$ is a 0-1 probabilistic automaton.*

These two propositions rely on the notion of *longest suffix resolvers* and how these resolvers interact with the layered structure. We provide full proofs in Section 3.4 in the appendix.

3.5 Some decision procedures in PTIME

Layered automata exhibit strong algorithmic properties. In this section, we show that determining whether a given layered automaton is consistent is decidable in polynomial time and that for consistent layered automata, both emptiness and language inclusion are also decidable in polynomial time. These algorithms exploit the strong acceptance characterisation of accepted languages. Complete constructions are detailed in Section 3.5 of the appendix.

Proposition 3.15. *It is decidable in polynomial time whether a given layered automaton is consistent.*

The algorithm checks if there is a pair $p, p' \in Q_{\geq 1}$ conflicting with the definition of consistency, that is, $\widehat{\mu}_1(p) = \widehat{\mu}_1(p')$ and there is a word w strongly accepted by p and strongly rejected by p'. The algorithm constructs a generalised Büchi game whose positions are pairs of states of the automaton. One player attempts to find a word w that is both strongly accepted by p and strongly rejected by p', thereby witnessing an inconsistency. The opponent tries to show that no such inconsistency exists. Since generalised Büchi games are decidable in polynomial time, the consistency check itself is decidable in polynomial time.

Proposition 3.16. *Given a consistent layered automaton $\mathcal{A}$, we can check in polynomial time whether $L(\mathcal{A}) = \emptyset$.*

Since we assume that all states in $\mathcal{T}_1$ are reachable, $L(\mathcal{A}) \neq \emptyset$ if and only if there is some state $p_0 \in Q_{\geq 1}$ in an even layer that strongly accepts some word. Once again the algorithm is to construct a game where the first player gives a word and tries to show that it is strongly accepted, and the second player aims to show that it is not. The game is a Büchi game this time.

Proposition 3.17. *Given two consistent layered automaton $\mathcal{A}$ and $\mathcal{A}'$, we can check in polynomial time whether $L(\mathcal{A}) \subseteq L(\mathcal{A}')$.*

This time we reduce this problem to solving a Rabin game of polynomial size and with 3 Rabin pairs. This allows to conclude, as solving Rabin games with a fixed number of Rabin pairs can be done in polynomial time (see e.g. [23, Thm. 3.1]).

3.6 Connections and comparison with other models

In this section, we discuss how layered automata relate to other models from the literature. Sometimes we refer to the minimal layered automaton for a language, which we introduce in Section 4. We leave for future work a thorough analysis on the size of layered automata compared to other representations of ω-regular languages; we include here just some remarks on this matter.

Deterministic, HD coBüchi, and alternating automata. Consistent layered automata are never larger than deterministic automata, though they can be exponentially smaller, as demonstrated for HD coBüchi automata [31, Thm. 1]. Indeed, minimal HD coBüchi automata are a special case of layered automata. By contrast, layered automata can be doubly exponentially larger than alternating parity automata. This double exponential gap occurs even for automata over finite words [14, Thm. 5.3] and extends to consistent layered automata since the first layer requires at least as many states as there are residuals of the language.

Muller languages. The reader familiar with Zielonka trees [43] and their parity automata [12, 18] may have noticed close resemblances between layered automata and these notions. Indeed, it is not difficult to see that the Zielonka tree of a Muller language L can be taken as the tree of a layered automaton recognising L. In this case, the associated alternating automaton $[\![\mathcal{A}]\!]$ is deterministic by pruning. Any such pruning produces the deterministic parity automaton of the Zielonka tree for the language L. We refer to [12, Section 4] for definitions on Muller languages and the parity automaton of a Zielonka tree.

Positional languages. In 2024, Casares and Ohlmann characterised which ω-regular languages are *positional*, that is, languages L such that the existential player can always play optimally using positional strategies in games with winning condition L [13]. In this characterisation, they use so-called *signature automata*. It turns out that signature automata are just layered automata with extra properties that ensure positionality (point (2) in [13, Thm. 3.1]).

Chains of coBüchi automata (COCOA). In [22], Ehlers and Schewe propose to represent an ω-regular language by a decreasing sequence of coBüchi languages $L_1 \supseteq L_2 \supseteq \cdots \supseteq L_d$,[4] where L_x are the words that are *not at home at level* $x - 1$ (we refer to [22, Def. 1, Thm. 7] for definitions[5]). A word w is in the language L represented by the chain if the maximal index x such that $w \in L_x$ is even.

It is straightforward to obtain a COCOA representation from a consistent layered automaton. For every layer x define an HD coBüchi automata from the transition system $\mathcal{T}_x$ by interpreting the existing transitions as 2-transitions of the coBüchi automaton, and for all missing transitions adding 1-transitions to all states of the corresponding residual class. Therefore, the size of a COCOA representation of a language is never bigger than the size of the minimal layered automaton for that language. But the minimal layered automaton may be exponentially larger than the minimal COCOA for a language, as illustrated by Example 3.29 in the appendix.

Since COCOA represent languages as Boolean combinations of coBüchi automata, it is not surprising that they can be smaller than representations by a single automaton. However, applying COCOA in practice requires constructing a history deterministic automaton from it, which is achieved by taking a product of all constituent coBüchi automata [20]. Consequently, the resulting alternating parity automaton may be exponentially larger than those obtained from layered automata. In Example 3.30 in the appendix, we demonstrate this phenomenon with a prefix-independent language (having a single residual), showing that the exponential blow-up is inherent to the product construction itself, not merely an artifact of handling multiple residuals.

One of the central definitions in [22] is the *natural color* of a word w with respect to a language L. We believe that the natural color is visible in the minimal layered automaton for L, namely it coincides with the maximal layer x in which w is ultimately safe.

Rerailing automata. In 2025, Ehlers introduced *rerailing automata* [19], a kind of automata with specific semantics: a word is accepted if the maximal priority over all its runs is even. Moreover, they must satisfy a semantic "rerailing property", inspired from the properties of minimal HD coBüchi automata. (We refer to [19] for precise definitions.)

We believe that a consistent layered automaton $\mathcal{A}$ can be transformed into a rerailing automaton by further saturating $[\![\mathcal{A}]\!]$. More precisely, whenever there is a transition $p \xrightarrow{a:x} [q]_x$, we can add transitions $p \xrightarrow{a:y} [q]_y$ for all $y < x$ without changing the language recognised by the automaton. The automaton obtained in this way has the rerailing property. Moreover, a word is accepted by $\mathcal{A}$ if and only if it is accepted with the semantics of rerailing automata after saturation.

[4] The definitions in [22] use 0 as minimal color, whereas we use 1 as minimal layer. As mentioned in [22], the definitions can easily be adapted to the setting with 1 as minimal color.

[5] We believe that in [22, Thm. 7] it should read "i.e., w is *not* at home in color $i - 1$".

For the other direction, it is unclear to us if rerailing automata are layered, or even simply HD alternating. However, it may be the case that the minimal rerailing automaton of a language and its minimal layered automaton are essentially the same object.

4 A canonical minimal layered automaton

Each ω-regular language admits a unique minimal consistent layered automaton, characterised by three properties: normality, centrality, and safe minimality. These properties extend those of minimal HD coBüchi automata [2] and are formally defined below.

Theorem 4.1. *Let $\mathcal{A}$ and $\mathcal{B}$ be two equivalent consistent layered automata that are normal, central and safe minimal. Then, $\mathcal{A}$ and $\mathcal{B}$ are isomorphic (as layered automata).*

Two layered automata $\mathcal{A}$, $\mathcal{B}$ are *isomorphic* if there is a bijective function between their states that preserves both the structure of the transition systems and the tree structure. That is, a function $\varphi\colon \mathcal{A} \to \mathcal{B}$ such that the restriction to each layer $\mathcal{T}_x$ is an isomorphism of transition systems and preserving the child relation given by the morphisms μ_x. (See Section 5.1 for further definitions on morphisms of layered automata.)

4.1 Normality, centrality and safe minimality

We define the three properties that identify minimal consistent layered automata.

Definition 4.2. A layered automaton is in *normal form* if for all $x \geq 2$ and $q \in Q_x$ the following two properties hold.

N1 If $q \xrightarrow{u}_x q'$ for some $u \in \Sigma^*$ then $q' \xrightarrow{v}_x q$ for some $v \in \Sigma^+$.

N2 For all $p \in \mu_x^{-1}(q)$, there is some u for which $q \xrightarrow{u}_x$ is defined in $\mathcal{T}_x$ but $p \xrightarrow{u}_{x+1} \bot$ in $\mathcal{T}_{x+1}$.

Being in normal form ensures that there are no evidently useless transitions, and moreover the priorities of transitions in $[\![\mathcal{A}]\!]$ cannot be lowered without modifying the recognised language. The first property means that each $\mathcal{T}_x$, for $x \geq 2$, is a union of non-trivial SCCs. This can be achieved just by removing transitions changing the SCC. A removal of such a transition leads to its priority being lowered to $x-1$ in $[\![\mathcal{A}]\!]$. This does not change the accepted language, as such a transition cannot appear infinitely often on a run without some other transition of priority $\leq x-1$ appearing infinitely often too.

The second property says that the safe languages of children are always strictly included in the safe language of the parent. Otherwise, the SCC of p does not strongly accept/reject anything. In the semantics automaton the property means that if $p \xrightarrow{a:\geq x}$, then there is u and a run $p \xrightarrow{u:x}$ from p producing x as minimal priority.

An *x-SCC* of a layered automaton is just an SCC of $\mathcal{T}_x$. The next property requires that no x-SCC can be simulated by another one. We define this formally below.

Definition 4.3. For $x \geq 2$, and $p, q \in Q_x$ we define $p \preccurlyeq_x q$ when $p \sim_{x-1} q$ (that is $\mu_{x-1}(q) = \mu_{x-1}(p)$), and $L_x(p) \subseteq L_x(q)$.

Definition 4.4. A layered automaton is *centralised* if for every $x \geq 2$ and for all states $p, q \in Q_x$, if $p \preccurlyeq_x q$ then p, q are in the same x-SCC.

Intuitively, if $p \preccurlyeq_x q$ are in different x-SCCs, then the x-SCC of p and all its descendants can be removed. Because $p \preccurlyeq_x q$, this removal does not change anything about words being strongly accepted/rejected in the parent SCC of p and q. Therefore, such states can be removed without changing the language of the automaton.

Finally, on each layer we can perform a similar minimisation construction as for finite automata by merging states with the same safe language (or the same language on layer 1). We define by induction on x the relation $p \approx_x q$, for $p, q \in Q_{\geq x}$:

$$p \approx_1 q \quad \text{iff} \quad L(p) = L(q), \quad \text{and}$$

$$p \approx_x q \quad \text{iff} \quad p \approx_{x-1} q \quad \text{and} \quad L_x(p) = L_x(q) \quad (\text{for } x > 1).$$

Recall that $q \sim_x p$ if $q, p \in Q_{\geq x}$ have the same ancestor at layer x in $\mathcal{A}$.

Definition 4.5. A layered automaton is *safe minimal* when for all $x \geq 1$ and states $p, q \in Q_x$: if $p \approx_x q$ then $p = q$,

The three properties normal, safe minimal, and centralised characterise minimal layered automata (as stated in Theorem 4.1).

4.2 Central sequences

The proof of Theorem 4.1 relies on the notion of a central sequence of a state p—a finite word that uniquely identifies p. This concept extends the synchronising property from [13, Sect. 4.2.2] and the notion of central sequence for coBüchi automata from the full version of [34]. The three properties defined in the previous subsection ensure that every state has a central sequence.

Let $\mathcal{A}$ be a layered automaton and $p \in Q_x$ an x-state. We say that $z_p \in \Sigma^+$ is a *central sequence* for p if:

- $p \xrightarrow{z_p}_x p$ in $\mathcal{T}_x$;
- for all $q \in Q_x$ with $q \sim_{x-1} p$, either $q \xrightarrow{z_p}_x p$ or $q \xrightarrow{z_p}_x \bot$;
- for all $q' \in Q_{x+1}$ with $q' \sim_{x-1} p$ we have $q' \xrightarrow{z_p}_{x+1} \bot$.

In $[\![\mathcal{A}]\!]$, this means that $p \xrightarrow{z_p:x} [p]_x$, and that from any other state q in $[p]_{x-1}$, either $q \xrightarrow{z_p:x} [p]_x$ or $q \xrightarrow{z_p:<x}$, meaning z_p produces a priority $< x$. Observe that the third condition is needed to ensure that the runs in question do not have priority $> x$.

We proof the next lemma in Lemma 4.7 in the appendix.

Lemma 4.6. *Let $\mathcal{A}$ be a normal, centralised and safe minimal layered automaton. Then, for every $x \geq 2$, every state $p \in Q_x$ admits a central sequence.*

Based on central sequences, the proof of Theorem 4.1 constructs an isomorphism φ between $\mathcal{A}$ and $\mathcal{B}$ layer by layer. It roughly works as follows. On layer 1 the transition systems of $\mathcal{A}$ and $\mathcal{B}$ are just residual equivalence for L, so they are isomorphic. Let p be a state of $\mathcal{A}$ with parent r. We already have defined the isomorphism for the layer of r. Take a central sequence z_p, and let q be a state reached in $\mathcal{B}$ by reading z_p from some child of $\varphi(r)$. The main part of the proof then shows that q can be taken as image of p. Full details can be found in Section 4.3 in the appendix.

5 Minimisation in polynomial time

This section presents a polynomial time minimisation procedure for consistent layered automata. More precisely, the procedure

constructs the unique minimal layered automaton appearing in Theorem 4.1. The procedure establishes that every consistent layered automaton contains a subautomaton admiting a surjective morphism into this minimal automaton, thereby proving canonicity. We formalise these notions through morphisms of layered automata, which we introduce in Section 5.1.

Theorem 5.1. *Let $\mathcal{A}$ be a consistent layered automaton. We can build in polynomial time an equivalent consistent layered automaton $\mathcal{A}_L$ that is normal, central and safe minimal.*

Moreover, $\mathcal{A}$ contains a subautomaton[6] *that admits a surjective strong morphism to $\mathcal{A}_L$. In particular, $\mathcal{A}_L$ has no more states nor leaf states than $\mathcal{A}$.*

Together with Theorem 4.1, this gives us minimality and canonicity of normal, central and safe minimal consistent layered automata as stated in Theorem 1.1.

5.1 Morphisms of layered automata

A *morphism* between two layered automata $\mathcal{A}$ and $\mathcal{A}'$ is a function $\varphi\colon Q_{\geq 1} \to Q'_{\geq 1}$ preserving both the structure of the transition systems and the ancestor relation in the tree representation. That is:

- $\varphi(q_{\text{init}}) = q'_{\text{init}}$,
- if $q \xrightarrow{a}_x q'$ in $\mathcal{T}_x$, then $\varphi(q) \xrightarrow{a}_y \varphi(q')$ in $\mathcal{T}'_y$, for $y = \mathit{layer}(\varphi(q))$, and
- if p is an ancestor of q in $\mathcal{A}$, then $\varphi(p)$ is an ancestor of $\varphi(q)$ in $\mathcal{A}'$.

We say that φ is a *strong morphism* if moreover it preserves the absence of transitions, that is: if $q \xrightarrow{a}_x \bot$ then $\varphi(q) \xrightarrow{a}_y \bot$, for $y = \mathit{layer}(\varphi(q))$.

Note that, while states may change layer via φ, some rigidity is imposed by the conditions. Notably, if $q \in Q_x$ is sent to $\mathcal{T}'_y$, then all states reachable from q in $\mathcal{T}_x$ are also sent to the layer y. In particular, due to our reachability assumption, all states in $\mathcal{T}_1$ are sent to $\mathcal{T}_1{}'$.

A layered automaton $\mathcal{A}'$ is a *subautomaton* of $\mathcal{A}$ if there exists an injective morphism $\varphi\colon \mathcal{A}' \to \mathcal{A}$.

Note that $\mathcal{A}'$ may be obtained not just by removing states from some layers in $\mathcal{A}$, but also by moving some SCCs from one layer to another.

A morphism of layered automata φ is an *isomorphism* if it is bijective and φ^{-1} is also a morphism. Note that in this case it is strong, and it preserves layers, that is, $\varphi|_{Q_x}$ is an isomorphism of transition systems from $\mathcal{T}_x$ to $\mathcal{T}_x'$.

5.2 Transformations on layered automata

The proof of Theorem 5.1 proceeds by transforming a given consistent layered automaton $\mathcal{A}$ through successive operations. The first establishes normality by ensuring properties N1 and N2: we separate SCCs to ensure N1, then apply a lowering operation to ensure N2. Subsequently, we quotient through the $\approx_x$ relation to obtain safe minimality, and finally we centralise the result. Crucially, when applied in this order, these operations preserve already-established properties. Each of these operations is computable in polynomial time.

5.2.1 Separating SCCs. In order to ensure property N1 we separate every $\mathcal{T}_x$ into disconnected, non-trivial SCCs. Let us fix $x > 1$ and suppose that in $\mathcal{T}_x$ there is a transition $p \xrightarrow{a}_x q$ of $\mathcal{T}_x$, such that the two states are not in the same SCC. The operation is simply to remove the transition $p \xrightarrow{a}_x q$ from $\mathcal{T}_x$ and all transitions mapping to it via $\widehat{\mu}_x$ in $\mathcal{T}_y$ for $y > x$. We also remove trivial SCCs that may have been generated (states in $\mathcal{T}_y$ with no outgoing transition) until there are no such states in the result. Let us denote the resulting automaton $\mathcal{A}^{sep}$.

Lemma 5.2. *Let $\mathcal{A}$ be a consistent layered automaton. Then, $\mathcal{A}^{sep}$ is consistent. Moreover, $L(\mathcal{A}^{sep}) = L(\mathcal{A})$, and $\mathcal{A}^{sep}$ is a subautomaton of $\mathcal{A}$.*

The proof of this lemma appears in Section 5.2.1 in the appendix.

After applying this operation we remove at least one transition. So, by repeating this operation at most $|Q_{\geq 1}|^2$ times, we obtain an equivalent consistent subautomaton satisfying N1.

5.2.2 Lowering. Assume $\mathcal{A}$ satisfies property N1. If N2 fails for a state q at level x, then q has a child p such that whenever $q \xrightarrow{u}_x$ is defined, we also have $p \xrightarrow{u}_{x+1}$ defined for all $u \in \Sigma^*$. Let S_{x+1} be the SCC of p in $\mathcal{T}_{x+1}$ and S_x the SCC of $q = \mu_x(p)$ in $\mathcal{T}_x$. We call S_x *covered* by S_{x+1} if for every $p' \in S_{x+1}$ and every transition $\mu_x(p') \xrightarrow{a}_x$, there is also a transition $p' \xrightarrow{a}_{x+1}$. When looking at $[\![\mathcal{A}]\!]$ this means that states mapping to S_{x+1} have no outgoing transitions of priority x. Our goal is to eliminate such covered SCCs.

To understand the lowering operation, recall the forest representation of layered automata (page 4). The operation replaces the subforest rooted at nodes S_x (layer x) with the subforest rooted at nodes of layer $x+2$ that map to S_{x+1}. More precisely, for each $y \geq x+2$, define S_y as the part of $\mathcal{T}_y$ induced by states mapping to S_{x+1}: $\{p \in \mathcal{T}_y : \widehat{\mu}_{x+1}(p) \in S_{x+1}\}$. Similarly, for $y \geq x$, define Z_y as the part of $\mathcal{T}_y$ mapping to S_x: $\{p \in Q_{\geq x} : \widehat{\mu}_x(p) \in S_x\}$. The new layered automaton $\mathcal{A}^{low}$ is obtained by replacing Z_y with S_{y+2}:

$$\mathcal{T}_y^{low} = \begin{cases} (\mathcal{T}_y - Z_y) \cup S_{y+2} & y \geq x, \\ \mathcal{T}_y & \text{otherwise.} \end{cases}$$

Example 5.3. Consider the example automaton shown on the left-hand side in Figure 3. Say we pick $S_x = \{p_2\}$ and $S_{x+1} = \{p_3\}$. Note that S_x is covered by S_{x+1}. The result is shown on the right-hand side of Figure 3. The parent relation is preserved for all states except for the ones in S_{x+2}, which is $S_4 = \{p_4, q_4\}$ in this case.

Lemma 5.4. *Let $\mathcal{A}$ be a consistent layered automaton satisfying N1. Then, $\mathcal{A}^{low}$ is consistent, and satisfies N1. Moreover, $L(\mathcal{A}^{low}) = L(\mathcal{A})$, and $\mathcal{A}^{low}$ is a subautomaton of $\mathcal{A}$.*

It can be verified that each lowering operation removes entire SCCs, so it preserves property N1. When there are no covered SCCs we obtain an equivalent subautomaton in normal form, namely, satisfying both N1 and N2. Finding a covered SCC and removing it can clearly be done in PTIME, so the whole normalization procedure can be done in PTIME too.

Full proofs appear in Section 5.2.2 of the appendix.

[6] We note that this subautomaton may not be consistent.

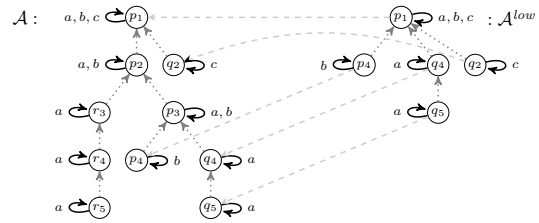

Figure 3: Illustration of the lowering operation for establishing N2 from Example 5.3.

5.2.3 Safe minimisation. The safe minimisation operation corresponds to a quotient operation on each layer. Namely, we replace each state p on layer x by $[p]_{\approx_x}$, the equivalence class of p with respect to the $\approx_x$ relation. We call the resulting automaton $\mathcal{A}^{\text{smin}}$. It can be shown that $\mathcal{A}^{\text{smin}}$ is a consistent layered automaton equivalent to $\mathcal{A}$, and that it is normal if $\mathcal{A}$ is normal. Moreover, the function $\varphi\colon \mathcal{A} \to \mathcal{A}^{\text{smin}}$ mapping a state p on level x to its equivalence class $[p]_{\approx_x}$ is a strong morphism, which is clearly surjective. See Section 5.3 in the appendix for details.

Lemma 5.5. *Let $\mathcal{A}$ be a normal and consistent layered automaton. Then, $\mathcal{A}^{\text{smin}}$ is safe minimal, normal, and consistent. Moreover, $L(\mathcal{A}) = L(\mathcal{A}^{\text{smin}})$, and $\mathcal{A}$ admits a surjective strong morphism to $\mathcal{A}^{\text{smin}}$.*

Observe that $\approx_x$ equivalence can be computed in PTIME as it amounts to language equivalence between two states in a deterministic safety automaton. Hence, the safe minimisation operation can be done in PTIME.

5.2.4 Centralisation. We now describe how to make an automaton centralised while preserving normalisation and safe minimality. It makes sense to write $S_1 \preccurlyeq_x S_2$ for two SCCs on layer x. This is because if there is some state in S_1 that is $\preccurlyeq_x$-smaller than some state in S_2 then every state in S_1 is $\preccurlyeq_x$-smaller than some state in S_2 (Lemma 5.13 in appendix). The *centralisation operation* removes from each level x all the x-SCCs that are not $\preccurlyeq_x$-maximal, as well as their descendants in the forest representation. We call $\mathcal{A}^{ctl}$ the resulting automaton.

Lemma 5.6. *Let $\mathcal{A}$ be a safe minimal, normal, and consistent layered automaton. Then, $\mathcal{A}^{ctl}$ is also safe minimal, normal and consistent. Moreover, $L(\mathcal{A}^{ctl}) = L(\mathcal{A})$, and $\mathcal{A}^{ctl}$ is a subautomaton of $\mathcal{A}$.*

The proof of this lemma appears in Section 5.4 in the appendix.

Every application of the centralisation operation removes some states. When the operation cannot be applied anymore, after at most $|Q_{\geq 1}|$ steps, we get an equivalent automaton that is centralised while still being normal, safe minimal, and consistent.

5.3 Summing up the proof of Theorem 5.1

Applying the polynomial-time transformations presented in the previous subsection, we obtain an automaton $\mathcal{A}_L$ that is normal, central and safe minimal, which proves the first part of Theorem 5.1.

We show the second part of this theorem, that is, that the initial automaton $\mathcal{A}$ contains a subautomaton admitting a surjective strong morphism to $\mathcal{A}_L$.

The above construction took 3 steps, producing the following sequence of automata:

$$\mathcal{A} \rightsquigarrow \mathcal{A}_{\text{normal}} \xrightarrow{\varphi} \mathcal{A}_{\text{safeMin}} \rightsquigarrow \mathcal{A}_{\text{central}} = \mathcal{A}_L.$$

Moreover, we have proven that these automata satisfy the following relations:

- $\mathcal{A}_{\text{normal}}$ is a subautomaton of $\mathcal{A}$,
- $\mathcal{A}_{\text{normal}}$ admits a surjective strong morphism to $\mathcal{A}_{\text{safeMin}}$.
- $\mathcal{A}_{\text{central}}$ is a subautomaton of $\mathcal{A}_{\text{safeMin}}$ obtained by removing subtrees of at levels ≥ 2 of the latter automaton.

It can be shown that $\mathcal{A}' = \varphi^{-1}(\mathcal{A}_{\text{central}})$ is a layered automaton that is a subautomaton of $\mathcal{A}_{\text{normal}}$ (see Lemma 5.16 in the appendix). Hence, also a subautomaton of $\mathcal{A}$. Since φ is a strong morphism its restriction to $\mathcal{A}'$ is also strong. Moreover, φ is surjective, as $\mathcal{A}'$ is the preimage of φ. This concludes the proof of Theorem 5.1.

6 Congruence-based characterisation

In this section, we provide an alternative characterisation of minimal layered automata based on congruences over tuples of finite words. This approach yields a direct, algebraic construction of the canonical minimal layered automaton for any ω-regular language. The congruence-based perspective complements the syntactic minimisation procedure presented in Section 5 and offers additional insight into the structure of layered automata through the lens of equivalences on sequences of finite words. For example, the maximal length of such sequences reflects the parity index of the language.

6.1 Definition of the congruence

Fix an ω-regular language L. As states on layer x we use classes of an equivalence relation $\equiv_L$ over x-tuples $\overline{u} = (u_1, \ldots, u_x) \in (\Sigma^*)^x$ of finite words. The relation $\equiv_L$ can be seen as a single equivalence relation over the set of all finite tuples (of length at least one) of finite words over Σ, in which equivalent tuples must have the same length. We will use notation like $(\overline{u}, u_{x+1})$ for $(u_1, \ldots, u_x)$ for readability. We consider two operations on such finite tuples:

***Concatenation*:** We define $\overline{u} \cdot u_{x+1} := (u_1, \ldots, u_{x-1}, u_x u_{x+1})$ as concatenation with a word $u_{x+1} \in \Sigma^*$ in the last component. For $x = 1$ the resulting tuple is $(u_1 u_2)$. We use this operation for defining the transition relation on the individual layers.

***Merging*:** Merging the last two components of $(u_1, \ldots, u_x, u_{x+1})$ is the operation of removing the last component and concatenating it to the second last component, resulting in the tuple $(u_1, \ldots, u_x u_{x+1})$. We use this operation for the morphism from layer $x + 1$ to layer x.

Before going into the formal definitions, let us look an example to get an intuition for the notions that we are going to define.

Example 6.1. For $\Sigma = \{a, b\}$ let $L \subseteq \Sigma^\omega$ consist of those words in which aba occurs finitely often, and a and b both occur infinitely often. The layered automaton constructed from equivalence classes of $\equiv_L$ is shown in Figure 4. The language is prefix independent, so

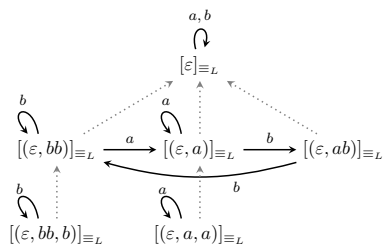

Figure 4: Congruence automaton from Example 6.1.

there is only one class on layer 1, and ε is used as representative in the first components.

Layer 2 is composed of equivalence classes of tuples of size 2. These are classes of tuples that are not equivalent to $\bot$, in symbols $(u_1, u_2) \not\equiv_L \bot$. These are tuples such that there is $w \in \Sigma^+$ with $u_1(u_2w)^\omega \in L$. The tuples for which no such extension exists are declared equivalent to $\bot$. So for our L we have $(u_1, u_2) \equiv_L \bot$ iff u_2 contains *aba*. We can now consider the safe language for each tuple: the set of finite words that can be appended to the last component without making the tuple $\bot$. On layer 2 our equivalence merges 2-tuples that have the same safe languages. In our example, this results in three classes on layer 2 that track the prefixes of *aba*, namely $[(\varepsilon, \varepsilon)]_{\equiv_L}$, $[(\varepsilon, a)]_{\equiv_L}$, $[(\varepsilon, ab)]_{\equiv_L}$. In consequence, on layer 2 we obtain a transition system that avoids the factor *aba*. Importantly, the representative (ε, bb) is used for the class $[(\varepsilon, \varepsilon)]_{\equiv_L}$. The intuitive reason is that *bb* is a word ensuring the following: concatenating it to any tuple (on layer 2) results in a tuple that is either $\bot$ or equivalent to (ε, bb). So in that sense, *bb* in the second component points to a unique class. This is captured by our definition of a pointed tuple, which is used for selecting those classes that are used as states.

Layer 3 is responsible for checking that infinitely many a's and b's occur, so words that contain only one of the two letters have to be safe on layer 3 (and are thus rejected because 3 is odd). Note that there is no state on layer 3 that maps to $[(\varepsilon, ab)]_{\equiv_L}$ on layer 2. The reason is that if $[(\varepsilon, ab)]_{\equiv_L}$ occurs infinitely often in a safe run on layer 2, then the word has to contain infinitely many a's and b's, so there is nothing to check on layer 3 in this case. Our definition of $\bot$ for tuples (u_1, u_2, u_3) of length 3 captures this by considering only words $w \in \Sigma^+$ that close a loop on the class of (u_1, u_2), written as $(u_1, u_2u_3w) \equiv_L (u_1, u_2)$.

We now proceed with the formal definitions, which generalise the definitions for the coBüchi case on pairs of words from [34].[7]

Bottom, safe language, and equivalence. Let $\overline{u} = (u_1, \ldots, u_x)$, $\overline{v} = (v_1, \ldots, v_x)$ be tuples of length $x \geq 1$ of finite words. For $x = 1$ we use the standard right congruence $u \sim_L v$ if $uw \in L \Leftrightarrow vw \in L$ for all $w \in \Sigma^\omega$. We define $(u_1) \not\equiv_L \bot$ for all u_1, and $(u_1) \equiv_L (v_1)$ iff $u_1 \sim_L v_1$. For tuples of length at least two, let

$$(\overline{u}, u_{x+1}) \equiv_L \bot \text{ if } \overline{u} \cdot u_{x+1} \equiv_L \bot, \text{ or}$$

$$\forall w \in \Sigma^+ : \overline{u} \cdot u_{x+1}w \equiv_L \overline{u} \Rightarrow [u_1 \cdots u_x(u_{x+1}w)^\omega \in L \;\Leftrightarrow\; x \text{ even}].$$

Note that the looping part $u_{x+1}w$ is non-empty because w is non-empty. For an infinite word $w \in \Sigma^\omega$ we define $(\overline{u}, w) \not\equiv_L \bot$ if $(\overline{u}, v) \not\equiv_L \bot$ for all prefixes v of w. The *safe language* of a tuple is:

$$L^s((\overline{u}, u_{x+1})) := \{w \in \Sigma^* \cup \Sigma^\omega \mid (\overline{u}, u_{x+1}w) \not\equiv_L \bot\}.$$

Based on that, let

$$(\overline{u}, u_{x+1}) \equiv_L (\overline{v}, v_{x+1}) \text{ if } \quad \begin{array}{l} \overline{u} \cdot u_{x+1} \equiv_L \overline{v} \cdot v_{x+1}, \text{ and} \\ L^s((\overline{u}, u_{x+1})) = L^s((\overline{v}, v_{x+1})) \end{array}$$

This is residual equivalence w.r.t. the safe language together with the condition that the tuples map to the same class on the previous layer. The class of a tuple is written $[\overline{u}]_{\equiv_L}$.

Example 6.2. Let $L = \text{parity}(1, 2, \ldots, d)$. Then, the tuples that are $\equiv_L \bot$ are those that have length $> d$ or that contain x in some component of index $> x$. Therefore, for every length $x \in \{1, \ldots, d\}$, there are exactly two equivalence classes: $[(\varepsilon, 1, 2, \ldots, x-1)]_{\equiv_L} \equiv_L \bot$, and $[(\varepsilon, \ldots, \varepsilon)]_{\equiv_L} \not\equiv_L \bot$.

The following lemma (Lemma 6.5 in the appendix) shows that $\equiv_L$ is a congruence w.r.t. to the operations that we consider on tuples (and hence $\equiv_L$ is preserved under arbitrary sequences of such operations).

Lemma 6.3. *Let $\overline{u}, \overline{v} \in (\Sigma^*)^x$, $a \in \Sigma$, $u_{x+1}, v_{x+1} \in \Sigma^*$. Then*
(1) $\overline{u} \equiv_L \overline{v} \Rightarrow \overline{u} \cdot a \equiv_L \overline{v} \cdot a$,
(2) $(\overline{u}, u_{x+1}) \equiv_L (\overline{v}, v_{x+1}) \Rightarrow \overline{u} \cdot u_{x+1} \equiv_L \overline{v} \cdot v_{x+1}$, and
(3) $\overline{u} \equiv_L \overline{v} \Rightarrow (\overline{u}, \varepsilon) \equiv_L (\overline{v}, \varepsilon)$.

Pointed. We now proceed with the definition of pointed tuples, which are used to select those classes of $\equiv_L$ that are actually used in the definition of the minimal automaton $\mathcal{A}_{\equiv_L}$. The definition of pointed is made such that it allows us to construct central sequences for the classes that contain pointed tuples. Consider a tuple $(\overline{u}, u_{x+1}) \not\equiv_L \bot$. It can be seen as the tuple that is reached from $(\overline{u}, \varepsilon)$ by reading u_{x+1}. The parent of $(\overline{u}, \varepsilon)$ on level x is $\overline{u}$. We now say that $(\overline{u}, u_{x+1})$ is pointed, if reading u_{x+1} from any $x+1$-tuple $(\overline{v}, v_{x+1})$ that has $[\overline{u}]_{\equiv_L}$ as parent on layer x, results in a tuple $(\overline{v}, v_{x+1}u_{x+1})$ that is either $\bot$ or equivalent to $(\overline{u}, u_{x+1})$. In other words, u_{x+1} uniquely points to the class of $(\overline{u}, u_{x+1})$ when starting from a child of $[\overline{u}]_{\equiv_L}$.

Formally, all tuples of length 1 are pointed, and $(\overline{u}, u_{x+1}) \not\equiv_L \bot$ is *pointed* if $\overline{u}$ is pointed and for all pointed $\overline{v}$ and all v_{x+1} with $\overline{v} \cdot v_{x+1} \equiv_L \overline{u}$:

$$\text{if } (\overline{v}, v_{x+1}u_{x+1}) \not\equiv_L \bot \text{ then } (\overline{v}, v_{x+1}u_{x+1}) \equiv_L (\overline{u}, u_{x+1}).$$

Example 6.4. Consider the alphabet $\Sigma = \{a, b, c\}$ and the language of all ω-words that contain at least one of the letters only finitely often. This language is prefix independent, so the first layer has only one class, and we only need to consider ε in the first component. Further, $(\varepsilon, u_2) \equiv_L \bot$ iff u_2 contains all letters, and $(\varepsilon, u_2) \equiv_L (\varepsilon, v_2)$ if the same letters occur in u_2 and v_2. All tuples of length more than 2 are $\bot$. For the automaton, we only need the classes $[(\varepsilon, ab)]_{\equiv_L}$, $[(\varepsilon, ac)]_{\equiv_L}$, and $[(\varepsilon, bc)]_{\equiv_L}$, which respectively check that c, b, or a occur finitely often. And these are precisely the

[7] In [34] for the coBüchi case all states of the congruence automaton are classes of length 2 tuples. With our definitions here applied to the coBüchi case, there might also be states that are classes of tuples of length 1. This flexibility of tuple length actually makes the definition simpler because it avoids a special treatment of ε as in [34].

classes that contain pointed tuples. Consider, for example, (ε, ab). Appending ab to any tuple of length 2 either leads to a tuple that is $\perp$ or to a tuple than contains only a and b in the second component, and hence is equivalent to (ε, ab). Whereas (ε, a) is not pointed because, for example (ε, aba) is not $\perp$ and not equivalent to (ε, a).

We show that for ω-regular languages, the number of $\equiv_L$-classes is finite. The first part of the proof shows that on each layer (for each fixed length of the tuples), there are finitely many $\equiv_L$-classes (using the fact that L can be accepted by a finite automaton). The second part shows that there are only finitely many layers that contain classes that are different from $\perp$. For tuples of length x that are not $\perp$, the proof constructs nested loops in a DPA for L that are alternating between rejecting and accepting. This bounds the length of these tuples to the parity index of the language. Full details are included in Lemmas 6.8 and 6.9 of the appendix.

Lemma 6.5. *If L is ω-regular, then $\equiv_L$ has finite index over $(\Sigma^*)^x$ for all $x \geq 1$. If L is accepted by a DPA with priorities in $[1, d]$, then $\overline{u} \equiv_L \perp$ for all $\overline{u} \in (\Sigma^*)^{d+1}$ (and hence also for all longer tuples).*

6.2 The automaton $\mathcal{A}_{\equiv_L}$

Let d be maximal such that there exists $\overline{u} \in (\Sigma^*)^d$ with $\overline{u} \not\equiv_L \perp$. By Lemma 6.5 this is the minimal d such that L is accepted by a $[1, d]$-DPA. The x-th layer of $\mathcal{A}_{\equiv_L}$ consist of the equivalence classes of the pointed tuples of dimension x, the transitions corresponding to concatenation in the last component. The morphism from layer $x + 1$ to layer x is obtained by merging the last two components. And the initial state is, as usual, the class of the empty word. Formally, we define the layered automaton $\mathcal{A}_{\equiv_L} = (q_0^{\equiv_L}, \mathcal{T}_1^{\equiv_L}, \mu_1^{\equiv_L}, \ldots, \mathcal{T}_{d-1}^{\equiv_L}, \mu_{d-1}^{\equiv_L}, \mathcal{T}_d^{\equiv_L})$ by

- $\mathcal{T}_x^{\equiv_L} = (Q_x^{\equiv_L}, \delta_x^{\equiv_L})$ with
  - $Q_x^{\equiv_L} = \{[\overline{u}]_{\equiv_L} \mid \overline{u} \in (\Sigma^*)^x \text{ pointed}\}$
  - $\delta_x^{\equiv_L}([\overline{u}]_{\equiv_L}, a) = \begin{cases} [\overline{u} \cdot a]_{\equiv_L} & \text{if } \overline{u} \cdot a \not\equiv_L \perp \\ \text{undefined} & \text{else.} \end{cases}$
- $q_0^{\equiv_L} = [\varepsilon]_{\equiv_L}$
- $\mu_x^{\equiv_L} : Q_{x+1}^{\equiv_L} \to Q_x^{\equiv_L}$ with $\mu_x^{\equiv_L}([(\overline{u}, u_{x+1})]_{\equiv_L}) = [\overline{u} \cdot u_{x+1}]_{\equiv_L}$.

Note that in the definition of $\delta_x^{\equiv_L}$ and $\mu_x^{\equiv_L}$ the result is independent of the chosen representative by Lemma 6.3(1) and (2). Moreover, one can show that if $\overline{u}$ is pointed then $\overline{u} \cdot a$ is pointed or $\overline{u} \cdot a \equiv_L \perp$. Further, a direct application of the definitions shows that $\mu_x^{\equiv_L}$ is a morphism. So $\mathcal{A}_{\equiv_L}$ is a layered automaton. The main result of this section is:

Theorem 6.6. *Let L be an ω-regular language. Then $\mathcal{A}_{\equiv_L}$ is the minimal layered automaton for L.*

For the proof that $\mathcal{A}_{\equiv_L}$ accepts L, the main part of the work is to show that the restriction to pointed classes is sufficient for accepting the correct language (Section 6.3 in the appendix). For the minimality proof, one establishes the properties from Theorem 4.1 for $\mathcal{A}_{\equiv_L}$ (Section 6.4 in the appendix). The properties N1 and safe minimality are easy consequences of the definitions. The main part of the proof shows how to construct central sequences for the states of $\mathcal{A}_{\equiv_L}$, which heavily relies on the definition of pointed. Having central sequences one can easily establish N2 and centralisation.

7 Conclusions

We have introduced layered automata, a model merging and generalising minimal HD coBüchi automata and Zielonka trees. This model provides canonical representations for ω-regular languages. We have characterised these representations via machine-independent congruences over tuples of finite words. Beyond this foundational characterisation, consistent layered automata exhibit strong algorithmic properties. Emptiness and inclusion testing are in PTIME for consistent layered automata. Furthermore, consistent layered automata can be minimised in PTIME, and the result is never bigger than the smallest equivalent deterministic automaton. These properties make consistent layered automata an attractive model of acceptors for ω-regular languages as they have good algorithmic properties and at the same time are both history deterministic and 0-1 probabilistic.

While we establish that canonical layered automata are minimal within the class of consistent layered automata (Theorem 1.1), we conjecture they are also minimal among all alternating history deterministic automata.

Conjecture 7.1. *Let $\mathcal{B}$ be a history deterministic alternating parity automaton. Then, there is an equivalent layered automaton $\mathcal{A}$ such that $[\![\mathcal{A}]\!]$ has no more states than $\mathcal{B}$.*

In particular, the minimal layered automaton of a language is minimal among all history deterministic alternating parity automata.[8]

If true, this conjecture would establish that layered automata provide a canonical and algorithmically efficient representation for all history deterministic automata.

Other future directions. Naturally, there are many aspects of the theory of layered automata itself that remain to be developed. More generally, we believe that layered automata offer a promising framework for broader practical and theoretical applications. A natural first direction is to provide efficient constructions for Boolean operations on languages and direct translations from logical formalisms such as Linear Temporal Logic to layered automata. A second avenue is to build the minimal layered automaton directly from COCOA representations using techniques from [20], thereby circumventing the full product construction discussed in Section 3.6. Finally, we envision applying layered automata to passive and active learning of omega-regular languages, extending the passive learning results for coBüchi automata [34].

[8] We have a draft of a proof for Conjecture 7.1 for the case of 2 priorities (Büchi and coBüchi automata). That is, the minimal HD coBüchi automaton from Abu Radi and Kupferman [2] is minimal among all HD alternating coBüchi automata.

References

[1] Bader Abu Radi and Rüdiger Ehlers. Characterizing the polynomial-time minimizable ω-automata. *CoRR*, abs/2504.20553, 2025. URL: https://doi.org/10.48550/arXiv.2504.20553, arXiv:2504.20553, doi:10.48550/ARXIV.2504.20553.

[2] Bader Abu Radi and Orna Kupferman. Minimization and canonization of GFG transition-based automata. *Logical Methods in Computer Science*, Volume 18, Issue 3:7587, 2022. doi:10.46298/lmcs-18(3:16)2022.

[3] Dana Angluin and Dana Fisman. Learning regular omega languages. *Theor. Comput. Sci.*, 650:57–72, 2016. doi:10.1016/J.TCS.2016.07.031.

[4] André Arnold. A syntactic congruence for rational omega-languages. *Theor. Comput. Sci.*, 39:333–335, 1985. doi:10.1016/0304-3975(85)90148-3.

[5] Guy Avni and Orna Kupferman. Stochastization of weighted automata. In *Mathematical Foundations of Computer Science 2015 - 40th International Symposium, MFCS 2015, Milan, Italy, August 24-28, 2015, Proceedings, Part I*, volume 9234 of *Lecture Notes in Computer Science*, pages 89–102. Springer, 2015. doi:10.1007/978-3-662-48057-1_7.

[6] Christel Baier, Marcus Grösser, and Nathalie Bertrand. Probabilistic ω–automata. *J. ACM*, 59(1), 2012. doi:10.1145/2108242.2108243.

[7] León Bohn and Christof Löding. Constructing deterministic parity automata from positive and negative examples. *TheoretiCS*, 3, 2024. doi:10.46298/THEORETICS.24.17.

[8] Udi Boker, Denis Kuperberg, Karoliina Lehtinen, and Michal Skrzypczak. On the succinctness of alternating parity good-for-games automata. In Nitin Saxena and Sunil Simon, editors, *FSTTCS*, volume 182 of *LIPIcs*, pages 41:1–41:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. doi:10.4230/LIPICS.FSTTCS.2020.41.

[9] Udi Boker and Karoliina Lehtinen. Good for games automata: From nondeterminism to alternation. In Wan J. Fokkink and Rob van Glabbeek, editors, *CONCUR*, volume 140 of *LIPIcs*, pages 19:1–19:16, 2019. doi:10.4230/LIPICS.CONCUR.2019.19.

[10] J. Richard Büchi. On a decision method in restricted second order arithmetic. *Proc. Internat. Congr. on Logic, Methodology and Philosophy of Science*, pages 1–11, 1962.

[11] Antonio Casares. On the minimisation of transition-based Rabin automata and the chromatic memory requirements of Muller conditions. In *CSL*, volume 216, pages 12:1–12:17, 2022. doi:10.4230/LIPIcs.CSL.2022.12.

[12] Antonio Casares, Thomas Colcombet, Nathanaël Fijalkow, and Karoliina Lehtinen. From Muller to parity and Rabin automata: Optimal transformations preserving (history) determinism. *TheoretiCS*, 3, 2024. doi:10.46298/THEORETICS.24.12.

[13] Antonio Casares and Pierre Ohlmann. Positional ω-regular languages. In *LICS*, pages 21:1–21:14. ACM, 2024. doi:10.1145/3661814.3662087.

[14] Ashok K. Chandra, Dexter C. Kozen, and Larry J. Stockmeyer. Alternation. *J. ACM*, 28(1):114–133, January 1981. doi:10.1145/322234.322243.

[15] E. M. Clarke, E. A. Emerson, and A. P. Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems*, 8(2):244–263, 1986. doi:10.1145/5397.5399.

[16] Thomas Colcombet. *Fonctions régulières de coût.* Habilitation (hdr), Université Paris Diderot – Paris 7, 2013.

[17] Thomas Colcombet and Christof Löding. The non-deterministic Mostowski hierarchy and distance-parity automata. In *Automata, Languages and Programming, 35th International Colloquium, ICALP 2008*, volume 5126 of *Lecture Notes in Computer Science*, pages 398–409. Springer, 2008. doi:10.1007/978-3-540-70583-3_33.

[18] Stefan Dziembowski, Marcin Jurdziński, and Igor Walukiewicz. How much memory is needed to win infinite games? In *LICS*, pages 99–110, 1997. doi:10.1109/LICS.1997.614939.

[19] Rüdiger Ehlers. Rerailing automata. *CoRR*, abs/2503.08438, 2025. doi:10.48550/ARXIV.2503.08438.

[20] Rüdiger Ehlers and Ayrat Khalimov. Fully Generalized Reactivity(1) Synthesis. In Bernd Finkbeiner and Laura Kovács, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, volume 14570, pages 83–102. 2024. doi:10.1007/978-3-031-57246-3_6.

[21] Rüdiger Ehlers and Ayrat Khalimov. A Naturally-Colored Translation from LTL to Parity and COCOA. (arXiv:2410.01021), 2024. arXiv:2410.01021, doi:10.48550/ARXIV.2410.01021.

[22] Rüdiger Ehlers and Sven Schewe. Natural Colors of Infinite Words. *LIPIcs, Volume 250, FSTTCS 2022*, 250:36:1–36:17, 2022. doi:10.4230/LIPICS.FSTTCS.2022.36.

[23] Nathanaël Fijalkow, C. Aiswarya, Guy Avni, Nathalie Bertrand, Patricia Bouyer, Romain Brenguier, Arnaud Carayol, Antonio Casares, John Fearnley, Paul Gastin, Hugo Gimbert, Thomas A. Henzinger, Florian Horn, Rasmus Ibsen-Jensen, Nicolas Markey, Benjamin Monmege, Petr Novotný, Pierre Ohlmann, Mickael Randour, Ocan Sankur, Sylvain Schmitz, Olivier Serre, Mateusz Skomra, Nathalie Sznajder, and Pierre Vandenhove. Games on graphs: From logic and automata to algorithms, 2025. To appear in Cambridge University Press. URL: https://arxiv.org/abs/2305.10546, arXiv:2305.10546.

[24] Ernst Moritz Hahn, Mateo Perez, Sven Schewe, Fabio Somenzi, Ashutosh Trivedi, and Dominik Wojtczak. Good-for-mdps automata for probabilistic analysis and reinforcement learning. In Armin Biere and David Parker, editors, *TACAS*, volume 12078 of *Lecture Notes in Computer Science*, pages 306–323, 2020. doi:10.1007/978-3-030-45190-5_17.

[25] Thomas A. Henzinger and Nir Piterman. Solving games without determinization. In *Computer Science Logic*, pages 395–410, 2006. doi:10.1007/11874683_26.

[26] Thomas A. Henzinger, Aditya Prakash, and K. S. Thejaswini. Resolving nondeterminism with randomness. In Pawel Gawrychowski, Filip Mazowiecki, and Michal Skrzypczak, editors, *MFCS*, volume 345 of *LIPIcs*, pages 57:1–57:18, 2025. doi:10.4230/LIPICS.MFCS.2025.57.

[27] Markus Holzer and Barbara König. On deterministic finite automata and syntactic monoid size. In *Developments in Language Theory, DLT 2002*, volume 2450 of *Lecture Notes in Computer Science*, pages 258–269. Springer, 2002. doi:10.1007/3-540-45005-X_22.

[28] Olivier Idir and Karoliina Lehtinen. Using games and universal trees to characterise the nondeterministic index of tree languages. In *52nd International Colloquium on Automata, Languages, and Programming, ICALP 2025*, volume 334 of *LIPIcs*, pages 160:1–160:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025. URL: https://doi.org/10.4230/LIPIcs.ICALP.2025.160, doi:10.4230/LIPICS.ICALP.2025.160.

[29] Nils Klarlund. A homomorphism concepts for omega-regularity. In *Computer Science Logic, 8th International Workshop, CSL '94, Kazimierz, Poland, September 25-30, 1994, Selected Papers*, volume 933 of *Lecture Notes in Computer Science*, pages 471–485. Springer, 1994. doi:10.1007/BFb0022276.

[30] Nils Klarlund, Anders Møller, and Michael I. Schwartzbach. MONA implementation secrets. *Int. J. Found. Comput. Sci.*, 13(4):571–586, 2002. doi:10.1142/S012905410200128X.

[31] Denis Kuperberg and Michał Skrzypczak. On determinisation of good-for-games automata. In *ICALP*, pages 299–310, 2015. doi:10.1007/978-3-662-47666-6_24.

[32] Orna Kupferman. Automata theory and model checking. In Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem, editors, *Handbook of Model Checking*, pages 107–151. Springer International Publishing, 2018. doi:10.1007/978-3-319-10575-8_4.

[33] Orna Kupferman, Shmuel Safra, and Moshe Y. Vardi. Relating word and tree automata. In *LICS*, pages 322–332, 1996. doi:10.1109/LICS.1996.561360.

[34] Christof Löding and Igor Walukiewicz. Minimal history-deterministic co-Büchi automata: Congruences and passive learning. In *LICS*, pages 431–443. IEEE, 2025. doi:10.1109/LICS65433.2025.00039.

[35] Oded Maler and Ludwig Staiger. On syntactic congruences for ω-languages. *Theoretical Computer Science*, 183(1):93–112, 1997. Formal Language Theory. URL: https://www.sciencedirect.com/science/article/pii/S030439759600312X, doi:10.1016/S0304-3975(96)00312-X.

[36] Zohar Manna and Amir Pnueli. A hierarchy of temporal properties. In Cynthia Dwork, editor, *PODC*, pages 377–410. ACM, 1990. doi:10.1145/93385.93442.

[37] Donald A. Martin. Borel determinacy. *Annals of Mathematics*, 102(2):363–371, 1975.

[38] Soumyajit Paul, David Purser, Sven Schewe, Qiyi Tang, Patrick Totzke, and Di-De Yen. Resolving nondeterminism by chance. In Patricia Bouyer and Jaco van de Pol, editors, *CONCUR*, volume 348 of *LIPIcs*, pages 32:1–32:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025. doi:10.4230/LIPICS.CONCUR.2025.32.

[39] Dominique Perrin and Jean-Éric Pin. Semigroups and automata on infinite words. In *Semigroups, formal languages and groups (York, 1993)*, volume 933, pages 49–72. Kluwer Acad. Publ., 1995.

[40] Amir Pnueli and Roni Rosner. On the synthesis of a reactive module. In *POPL*, page 179–190, 1989. doi:10.1145/75277.75293.

[41] Sven Schewe. Beyond hyper-minimisation—minimising DBAs and DPAs is NP-complete. In *FSTTCS*, volume 8, pages 400–411, 2010. doi:10.4230/LIPIcs.FSTTCS.2010.400.

[42] Thomas Wilke. An Eilenberg theorem for infinity-languages. In *ICALP*, volume 510, pages 588–599, 1991. doi:10.1007/3-540-54233-7_166.

[43] Wiesław Zielonka. Infinite games on finitely coloured graphs with applications to automata on infinite trees. *Theoretical Computer Science*, 200(1-2):135–183, 1998. doi:10.1016/S0304-3975(98)00009-7.